RAG Assignment: Baseline vs. Re-Ranking

Yogendra Patel 2025201098

19 October 2026

Code: `https://github.com/CReativeRebe1/RAG_LMA_Assignment1` (notebook `rag_baseline_vs_rerank.ipyn`

Setup

- **Document:** Digital Personal Data Protection Act, 2023 (official PDF, 21 pages). I split it by section and sub-section into 105 chunks of at most 200 tokens. The penalty table at the end (the Schedule) was rebuilt row by row so each fine stays next to its rule.
- **Bi-encoder:** `all-MiniLM-L6-v2`. Chunk vectors are stored in a FAISS index and compared with cosine similarity.
- **Cross-encoder:** `cross-encoder/ms-marco-MiniLM-L-6-v2`.
- **LLM:** `openai/gpt-oss-120b` on Groq, temperature 0. Both pipelines use the same prompt. It tells the model to answer only from the 3 chunks given, and to say so if they do not have the answer.
- **Test set:** 32 questions in everyday language. For each one I marked which chunk(s) of the Act really answer it.

1 Architecture Comparison

A **bi-encoder** turns the query into one vector and each chunk into one vector, separately. The score is just how close the two vectors are (cosine similarity). Since a chunk never sees the query, all chunk vectors can be made once and saved. A search then only needs to encode the query and look up the nearest vectors, which is very fast. The problem is that a whole chunk is squeezed into one vector before we know the question. Small but important words like "not" or "except" get lost, and a chunk that shares many words with the query can score high even if it does not answer it.

A **cross-encoder** reads the query and the chunk together as one input. Attention works across both texts, so every word of the query can be compared with every word of the chunk, and the model gives one score for the pair. This judges much better whether a chunk really answers the question. But nothing can be prepared in advance: the model has to run once for every (query, chunk) pair, at the moment the question is asked.

2 Qualitative & Rank Shift Analysis

Rank changes

Table 1 shows the rank of the correct chunk before and after re-ranking. Only the top 3 go to the LLM.

Question	Failure type	Correct chunk	Rank	In top 3?
Q09 Must the DPO be in India?	Keyword trap	Chunk 36 (s.10(2))	#5 → #1	no → yes
Q13 Can I take back my permission?	Keyword trap / synonym	Chunk 17 (s.6(4))	#4 → #1	no → yes
Q21 Can a company refuse service if I won't share extra data?	Negation / limit	Chunk 14 (s.6(1))	#4 → #2	no → yes
Q25 App unused for years, must data be deleted?	Keyword trap / synonym	Chunk 30 (s.8(8))	#5 → #3	no → yes
Q01 Max fine for weak security?	Penalty in another place	Chunk 103 (Schedule)	#3 → #1	yes → yes
Q03 Fine for not reporting a breach?	Penalty in another place	Chunk 103 (Schedule)	#3 → #1	yes → yes
Q27 Must the government erase my data?	Exception	Chunk 52 (s.17(4))	#6 → #4	no → no
Q12 Can I pretend to be someone else?	Negation	Chunk 45 (s.15)	#2 → #7	yes → no

Table 1: Rank of the correct chunk: baseline → re-ranked.

Why the baseline went wrong:

- **Keyword trap (Q09):** the baseline's top 3 were s.16 (data outside India), s.17(1) ("...any person based in India") and s.3 (territory of India). They only share the word "India" with the question. The actual rule ("the DPO shall be based in India") was at rank 5.
- **Different words, same meaning (Q13):** the question says "take back my permission", the Act says "withdraw her consent". The baseline picked chunks about deleting data and Consent Managers instead of s.6(4).
- **Penalty in another place (Q01, Q03):** the duty is written in s.8, but the fine amount is in the Schedule at the end. The baseline put general rule chunks (s.8, s.27, s.33) above the Schedule. The cross-encoder moved the Schedule to rank 1.
- **Negation / limit (Q21):** the answer is a limit (consent must be "limited to such personal data as is necessary"). The baseline returned other consent and deletion chunks instead.

Re-ranking is not always better. In **Q12** the cross-encoder pushed the s.15 chunk ("not to impersonate another person") from rank 2 down to rank 7. It seems to have matched "giving my personal data" with the consent chunks and did not see that "pretend to be someone else" means "impersonate".

In 8 of the 32 questions the correct chunk was not in the top 10 at all (for example Q11 about civil courts and Q15 about what happens to data rights after death). Re-ranking cannot fix these, because it only reorders the 10 chunks the bi-encoder found.

Pipeline	Right chunk at rank 1	Right chunk in top 3	Right chunk in top 10	MRR
Baseline (bi-encoder)	10/32 (0.31)	18/32 (0.56)	24/32 (0.75)	0.45
Re-ranked (cross-encoder)	14/32 (0.44)	21/32 (0.66)	24/32 (0.75)	0.55

Table 2: Retrieval results over all 32 questions. MRR is the average of 1/rank of the correct chunk (1 means it is always first, 0 means it is never in the top 10). The top-10 column is the same for both, because both use the same 10 chunks.

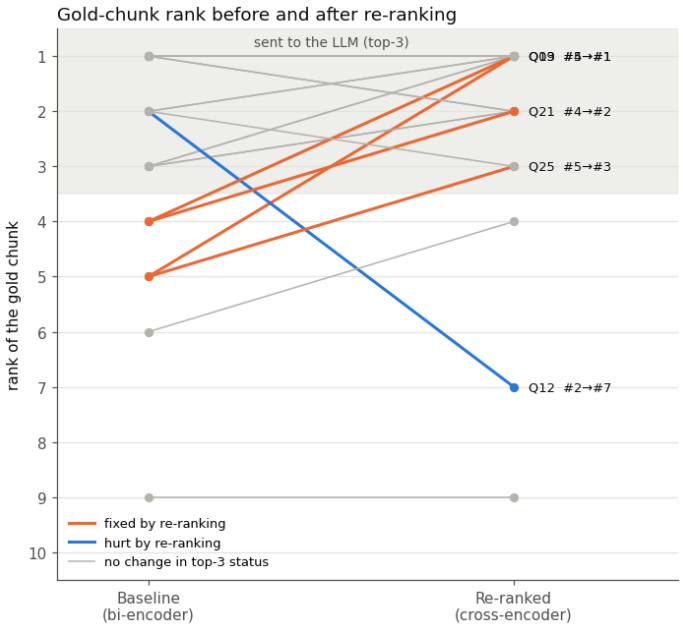


Figure 1: Rank of the correct chunk before and after re-ranking (one line per question).

Answer 1 vs. Answer 2

- **Q09:** Answer 1 said the chunks do not answer the question, so the user gets nothing. Answer 2 said "Yes, the DPO must be based in India" and cited the right chunk.
- **Q13:** Answer 1 said yes, but it was built from the Consent Manager and deletion chunks. It **left out an important part**: consent can be withdrawn at any time, as easily as it was given, and anything done before stays legal. Answer 2 had s.6(4) and gave the full rule.
- **Q21:** Answer 1 refused. Answer 2 correctly said consent must be limited to the data needed for the purpose.
- **Q25:** Answer 1 said "yes, delete" by guessing from the general deletion rule (s.8(7)) and **missed the real rule** (s.8(8): if you do not use the service for a set period, the purpose counts as over). Answer 2 had that chunk at position 3 but still refused, so here the LLM was the weak step, not retrieval.
- **Q12 (re-ranking made it worse):** Answer 1 was correct ("you must not impersonate another person"). Answer 2 refused.
- **Q20:** both had s.33(2) first, but re-ranking dropped the next chunk that holds factors (f) and (g), so Answer 2 **left out two of the seven factors** used to decide a fine.

I read all 64 answers and checked each one against the Act:

Result	Answer 1 (baseline)	Answer 2 (re-ranked)
Correct	17	16
Refused ("the chunks do not answer this")	11	13
Left out an important part	2	1
Gave a wrong rule	2	2
Made up facts (hallucination)	0	0

Table 3: My check of all 32 answers from each pipeline.

What I observed:

- When the correct chunk was missing, the baseline usually **refused** instead of making things up, because the prompt lets it say the chunks do not have the answer. When it answered anyway, it

either **left out a part** (Q13, Q25) or **gave a wrong rule** (Q26: it said formal consent is still needed, but s.7(a) allows data you hand over yourself to be used without it).

- Re-ranking clearly improves retrieval: the right chunk reached the top 3 in 21 questions instead of 18. It moved the right chunk into the top 3 in 4 questions and pushed it out in 1. It fixed the answers for Q09, Q13 and Q21.
- The final answers are about equally good (17 vs 16 correct). The gains were cancelled by Q12 (chunk pushed out), Q20 (a needed second chunk dropped) and two cases (Q03, Q25) where the right chunk was given but the LLM still refused. So better ranking helps, but on its own it did not make the answers better. With only 32 questions, the difference is also small.

3 Latency Profiling

Each step was timed on its own, 30 times per question after a warm-up run, on a laptop GPU (RTX 3050) and on the CPU.

	GPU (ms)		CPU (ms)	
Step	median	slowest 5%	median	slowest 5%
Encode the query (bi-encoder)	6.98	8.28	15.88	19.40
FAISS search for top 10	0.05	0.06	0.06	0.07
Cross-encoder on 10 chunks	27.02	28.19	329.80	375.55
Baseline retrieval (total)	**7.03**	8.33	**15.95**	19.46
Re-ranking pipeline (total)	**34.14**	35.94	**346.15**	393.39

Table 4: Retrieval time per question. "Slowest 5%" is the 95th percentile: 95% of runs were faster than this.

Re-ranking makes retrieval about 5 times slower on the GPU (+27 ms) and about 22 times slower on the CPU (+330 ms). The LLM call is the same for both pipelines and takes about 5 seconds (median of 63 calls), so the extra re-ranking time is small next to the total time to get an answer.

4 Scaling Question

A cross-encoder cannot prepare anything in advance, because its score depends on the query and the chunk together. So using it on the whole collection means running the model once for every chunk, for every question. The time grows directly with the number of chunks. A bi-encoder encodes all chunks once beforehand, so for a new question it only encodes the question and searches the saved vectors, which stays fast even for very large collections.

I measured this on the GPU:

Chunks in collection	Cross-encoder on every chunk	Bi-encoder + FAISS search
10	31 ms	–
1,000	2.2 s	7.6 ms
100,000	3.6 min (estimated)	37 ms
1,000,000	36 min (estimated)	–

Table 5: Time per question as the collection grows. The cross-encoder was measured on 10, 100 and 1000 chunks; larger sizes are estimated from the time per chunk.

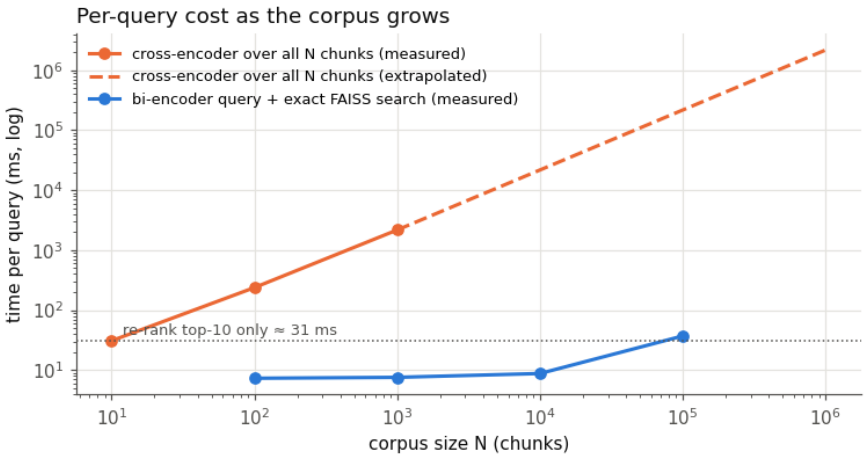


Figure 2: Time per question vs. collection size (log scale).

This is why we use two stages. The fast bi-encoder first picks a short list of 10 chunks from the whole collection, and its job is not to miss the right one. The slow but accurate cross-encoder then only sorts those 10, and its job is to put the right one first. Sorting 10 chunks takes about 27 ms, while running the cross-encoder on a million chunks would take more than half an hour for a single question.